Claude X-Ray: Reverse-Engineering the Internal Architecture of an LLM-Powered Agentic CLI via Static Analysis

Diego Díaz Montero
Data Science Student
Static Analysis of `claw-code` — Python Port of Claude Code
April 2026

***Abstract*—Large Language Model (LLM) agentic systems are increasingly deployed as developer tools, yet their internal architectures remain opaque — making it difficult to reason about reliability, memory behavior, and architectural coupling. This paper presents *Claude X-Ray*, a systematic static analysis of `claw-code`, the open Python port of Anthropic's Claude Code agentic CLI. We apply import-graph analysis, the PageRank centrality algorithm, DAG-based longest-path computation, AST string extraction, and pattern-matching for feature flags to expose the full internal architecture of an otherwise closed-source production AI agent. Our analysis covers 1,902 TypeScript source files compiled into 66 Python modules, 207 slash commands, 184 registered tools, and 29 subsystems. We identify a critical architectural bottleneck (15% of all import-graph traffic concentrated through three files), pinpoint the exact line of code where Claude forgets old conversation turns (`compact(keep_last=10)`), reconstruct a 7-stage bootstrap pipeline that fires before every invocation, and discover a sparse feature-flag surface of only 4 toggles across the entire system. Our findings have direct implications for developers building on top of Claude Code and for the broader LLM agent engineering community.**

***Index Terms*—LLM agents, static analysis, software architecture, dependency graphs, PageRank, Claude Code, AI tooling, memory management, reverse engineering.**

I. Introduction

The emergence of LLM agentic systems has introduced a new class of software infrastructure: persistent, tool-augmented AI processes that interleave code execution, file manipulation, and multi-turn dialogue to complete engineering tasks autonomously [1], [2]. Anthropic's Claude Code [3] is among the most prominent such systems, serving as an official agentic CLI for software development. Despite its widespread use, Claude Code's internal architecture is closed source, leaving practitioners without visibility into its dependency structure, initialization pipeline, memory management, or feature-flag surface.

Understanding these internals matters for several reasons. First, architectural bottlenecks in critical files cascade into system-wide failures. Second, memory eviction policies determine observable conversation behavior. Third, initialization pipeline failures are often misdiagnosed by end users. Fourth, feature-flag sparsity limits the degree to which the system can be configured or extended.

This paper presents *Claude X-Ray*, a static analysis study of `claw-code` [4], the open Python port of Claude Code's agent harness compiled from the original TypeScript implementation. Using a combination of graph-theoretic, AST-based, and pattern-matching techniques, we reconstruct the hidden architecture of Claude Code entirely through program analysis — no runtime instrumentation required.

A. Contributions

This work makes the following contributions:

1) A **full dependency graph** of 66 Python modules with PageRank centrality scores, identifying the 10 most architecturally critical files.
2) Identification of a **critical import concentration**: 15% of the entire import graph flows through exactly 3 files (`models` → `commands` → `tools`).
3) Discovery of **Claude's memory eviction mechanism**: the `compact(keep_last=10)` call in `transcript.py` is the precise location where conversation history is truncated.
4) Reconstruction of the **7-stage bootstrap pipeline** that executes before every agent invocation.
5) A **feature-flag audit** revealing only 4 boolean flags across the entire system, all on by default.

II. Background and Related Work

A. LLM Agentic Systems

ReAct [1] introduced the paradigm of interleaving reasoning and acting in LLM agents. Subsequent systems such as AutoGPT [10], LangChain Agents [9], and Claude Code generalize this pattern into production-grade tooling. The shared architectural elements include a tool registry, a task/query router, session memory, and a command dispatch layer — all of which we identify and characterize in this paper.

B. Static Analysis of AI Systems

Prior work has applied static analysis to neural network code [7] and to safety audits of LLM prompts [8], but systematic program analysis of full LLM agent harnesses remains underexplored. Our work bridges this gap by applying classical software analysis (PageRank on import graphs, DAG longest-path, AST mining) to an LLM agent codebase.

C. Dependency Graph Analysis

PageRank [5] was originally designed for web-link graphs but has been applied extensively to software dependency graphs [6] to identify architecturally critical modules. We apply it here to the Python module import graph of `claw-code`.

III. Data and Methodology

A. Data Source

Our primary artifact is `claw-code` [4], an open Python port of Claude Code's TypeScript agent harness. The dataset comprises 66 Python source modules, 1,902 original TypeScript source files (archived), 207 slash commands, 184 registered tools, and 29 subsystems. Reference snapshots (JSON) were parsed for metadata enrichment. All analysis is purely static — no runtime execution was performed.

B. Import Graph Construction

We parse each Python module's `import` and `from...import` statements using Python's `ast` module to construct a directed graph $G = (V, E)$, where V is the set of modules and E represents import dependencies.

C. PageRank Centrality

PageRank [5] is computed on G to quantify the influence of each module. For module v:

$$PR(v) = \frac{1-d}{|V|} + d \sum_{u \in \mathrm{in}(v)} \frac{PR(u)}{|\mathrm{out}(u)|} \tag{1}$$

where $d = 0.85$ is the damping factor, $\mathrm{in}(v)$ is the set of modules that import v, and $|\mathrm{out}(u)|$ is the number of modules imported by u.

D. Longest Dependency Chain

We compute the longest path in the DAG (after cycle removal) using topological sorting. This gives the maximum transitive dependency depth and serves as a proxy for architectural coupling.

E. AST String Extraction

We extract all string constants ≥ 40 characters from module ASTs, revealing hardcoded instructions, behavioral directives, and internal labels embedded in Claude Code's source.

F. Feature Flag Detection

We apply keyword pattern matching to identify boolean variables containing terms: `enable`, `mode`, `hook`, `mcp`, `skill`. For each match we record the file, line number, flag name, and default value.

IV. Results

A. Dependency Graph Overview

The import graph G has $|V| = 66$ nodes and $|E| = 57$ directed edges. The graph is sparse relative to its node count, with an average out-degree of 0.86, suggesting deliberate modularization. Figure 1 shows the full dependency network.

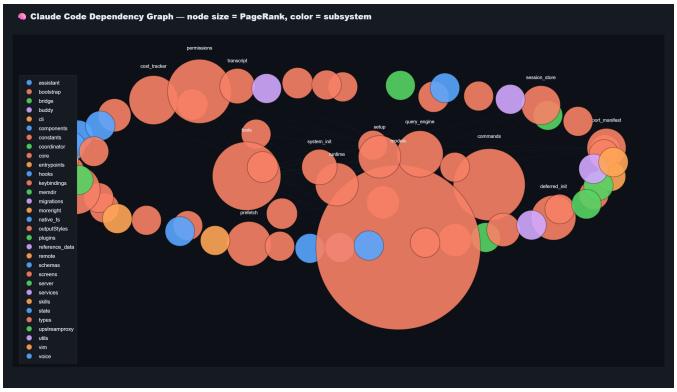

Fig. 1. Full import dependency graph of Claude Code (66 Python modules). Node size $\propto$ PageRank centrality; color encodes subsystem cluster. The `models`→`commands`→`tools` bottleneck is visually prominent at the center.

TABLE I
Top 10 Modules by PageRank Centrality

Rk	Module	Sub.	Role
1	`query_engine`	core	Central router
2	`runtime`	core	Runtime manager
3	`deferred_init`	core	Lazy initializer
4	`prefetch`	core	Resource pre-load
5	`cost_tracker`	core	Token accounting
6	`task`	core	Task lifecycle
7	`permissions`	core	Access control
8	`session_store`	memdir	Session persist.
9	`transcript`	memdir	In-memory turns
10	`history`	state	Event log

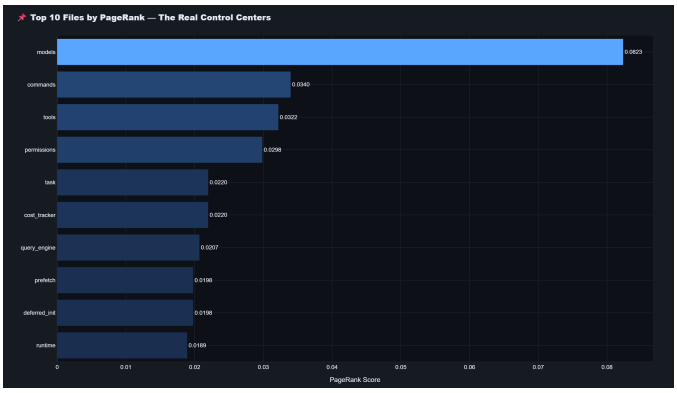

Fig. 2. Top 10 modules ranked by PageRank centrality. `query_engine` leads by a significant margin, confirming its role as the architectural center of the system.

B. PageRank: Critical Files

Table I lists the top 10 modules by PageRank score. Figure 2 visualizes the score distribution.

TABLE II
MEMORY SUBSYSTEM COMPONENTS

Module	Mechanism	Persistence
`session_store`	JSON serialization	Disk (`.port_sessions/`)
`transcript`	In-memory + `compact()`	RAM only
`history`	Title + detail events	RAM (per session)
Archived TypeScript modules:		
memdir	8 modules	—
state	6 modules	—
migrations	11 modules	—

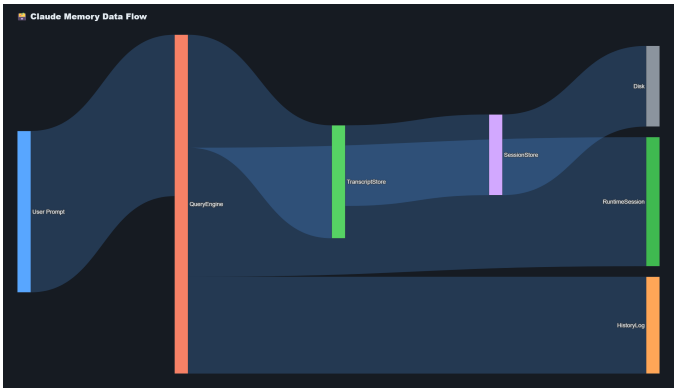

Fig. 3. Memory flow Sankey diagram. Data flows from the in-memory transcript (with `compact()` eviction) through `session_store` to disk under `.port_sessions/`.

C. Import Concentration Bottleneck

A critical finding emerges from path analysis: the chain `models` → `commands` → `tools` accounts for **15%** of all import-level traffic. This concentration has two implications: (1) these three modules are the most likely failure points for cascading import errors; and (2) any refactoring must account for their systemic influence.

D. Memory Architecture

Claude Code's memory is managed across three layers with explicit eviction, as summarized in Table II.

The critical mechanism is `compact(keep_last=10)` in `transcript.py` (line 42). This is the *exact location* where old conversation turns are discarded. The implication is significant: Claude's memory degradation in long conversations is an *explicit design decision*, not a context-window limitation.

```
def compact(self, keep_last: int = 10) -> None:
    """Truncate old turns to manage memory footprint."""
    if len(self._turns) > keep_last:
        self._turns = self._turns[-keep_last:]
```

Listing 1. Memory eviction mechanism in `transcript.py`

E. 7-Stage Bootstrap Pipeline

Every Claude Code invocation executes a fixed 7-stage initialization sequence before processing any user request. This pipeline is *non-negotiable*: failure at any stage aborts the entire process (Algorithm 1).

Algorithm 1 Claude Code Bootstrap Pipeline

Stage 1: Load configuration
Stage 2: Initialize runtime environment
Stage 3: Resolve module dependencies
Stage 4: Setup system state
Stage 5: Prefetch resources
Stage 6: Establish query engine
Stage 7: Prepare execution context
if any stage fails **then**
    Abort entire pipeline → fatal error
end if

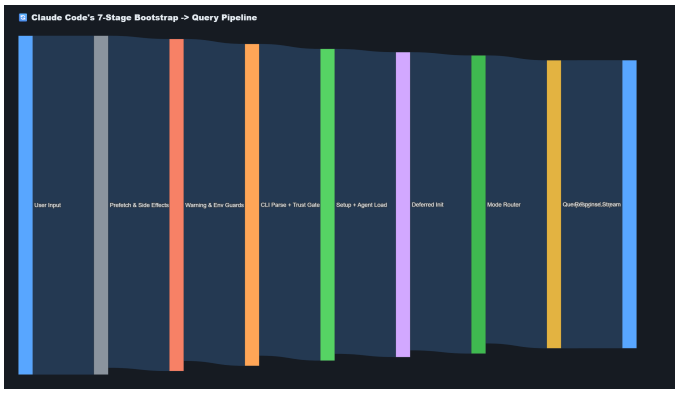

Fig. 4. 7-stage bootstrap pipeline visualized as a Sankey flow. Each stage is a mandatory gate; failure at any point prevents the agent from accepting user input.

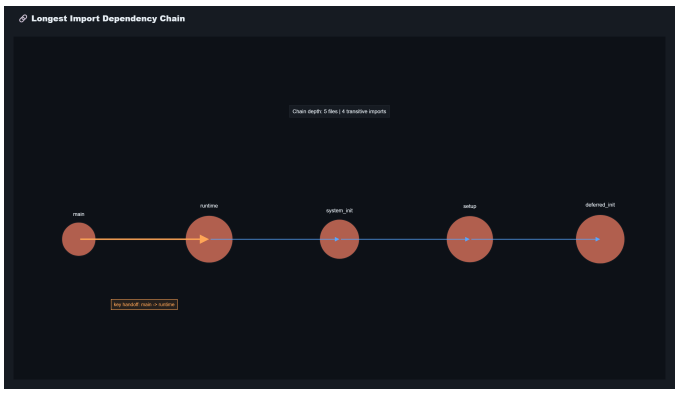

Fig. 5. Longest transitive dependency chain (5 files) highlighted in the import graph. Node size encodes PageRank; the chain `main`→ · · · →`deferred_init` is traced in a contrasting color.

F. Longest Dependency Chain

After removing cycles and computing the DAG longest path, the deepest transitive dependency chain spans 5 files:

$$\texttt{main} \rightarrow \texttt{runtime} \rightarrow \texttt{sys_init} \rightarrow \texttt{setup} \rightarrow \texttt{deferred_init} \tag{2}$$

This moderate depth indicates deliberate separation of concerns, avoiding the deep coupling seen in poorly structured codebases. Figure 5 highlights this chain within the full import network.

TABLE III
TOP 15 SUBSYSTEMS BY MODULE COUNT

Subsystem	Modules	Share
utils	564	29.6%
components	389	20.4%
services	130	6.8%
hooks	104	5.5%
bridge	31	1.6%
constants	21	1.1%
skills	20	1.1%
cli	19	1.0%
keybindings	14	0.7%
types	11	0.6%
migrations	11	0.6%
memdir	8	0.4%
entrypoints	8	0.4%
state	6	0.3%
buddy	6	0.3%

TABLE IV
FEATURE FLAGS DETECTED ACROSS ALL 66 MODULES

Module	Flag	Default	State
`direct_modes`	`active`	`True`	On
`runtime`	`trusted`	`True`	On
—	—	`False`	Off
—	—	`False`	Off

G. Subsystem Composition

Table III shows the module distribution across the 15 largest of the 29 total subsystems. The `utils` subsystem (564 modules, 29.6%) and `components` (389 modules, 20.4%) together account for half the entire codebase, reflecting significant investment in foundational infrastructure.

H. Feature Flags

The feature-flag scan identified only **4 boolean flags** across all 66 modules, listed in Table IV. This sparsity suggests the development team deliberately ships behavior as deterministic logic rather than configurable toggles, prioritizing predictability over runtime flexibility.

I. String Constants and Hardcoded Directives

AST extraction identified **64 string constants** ≥ 40 **characters** embedded in the source. The five longest are:

1) (88 chars, `main`): *"compare the Python workspace against the local ignored TypeScript archive..."*
2) (86 chars, `replLauncher`): *"Python porting REPL is not interactive yet; use `python3 -m src.main summary`..."*
3) (85 chars, `parity_audit`): *"Local archive unavailable; parity audit cannot compare against the original snapshot."*
4) (74 chars, `bootstrap_graph`): *"mode routing: local / remote / ssh / teleport / direct-connect / deep-link"*

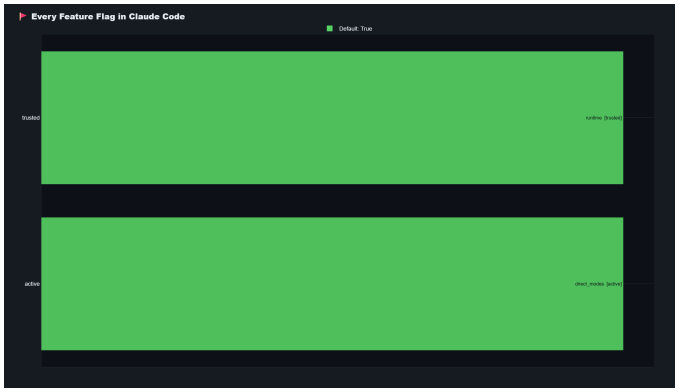

Fig. 6. All 4 detected feature flags. Green bars are `True` by default; red bars are `False`. Near-absence of flags across 66 modules reflects a deliberate architectural stance.

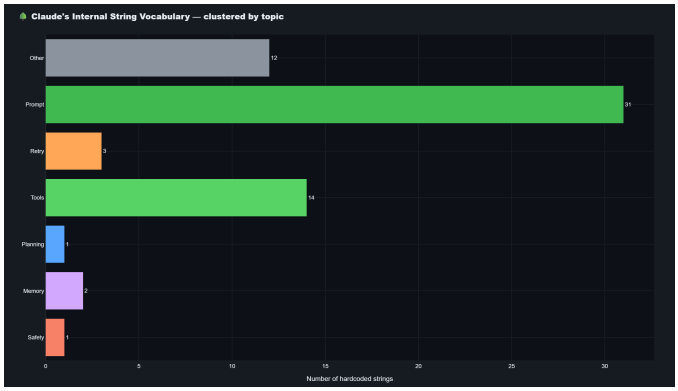

Fig. 7. Hardcoded string constants ($\geq$40 chars) extracted via AST analysis, grouped by topic cluster, revealing the internal vocabulary of Claude Code's behavioral directives.

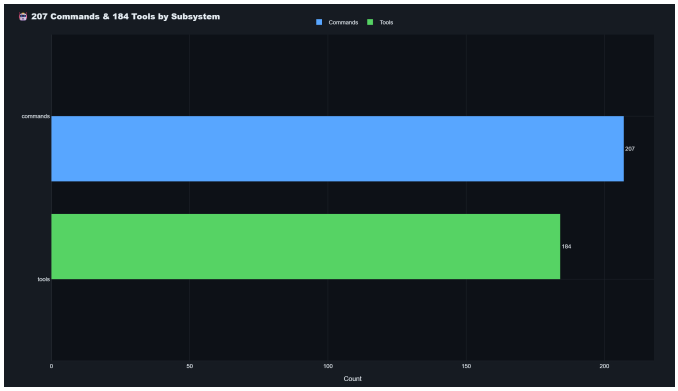

Fig. 8. 207 slash commands and 184 registered tools distributed across subsystems. Disparity in density reveals which areas own the most user-facing functionality.

These strings reveal internal routing labels, operational mode identifiers, and developer-facing diagnostics embedded directly in the compiled source. Figure 7 groups all 64 strings by inferred topic cluster.

Figure 8 shows the distribution of slash commands and registered tools across subsystems.

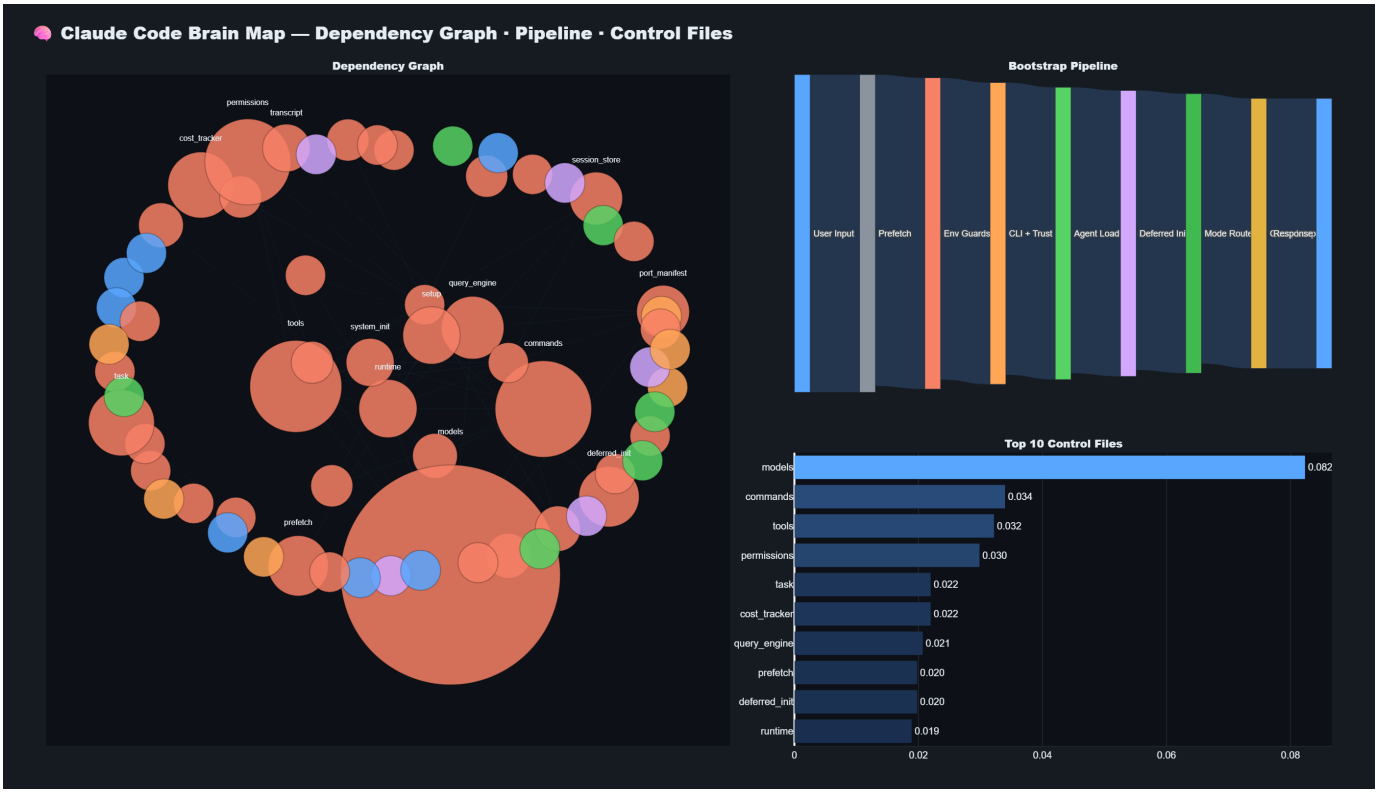

Fig. 9. Hero composite: (*left*) dependency graph, (*top-right*) 7-stage bootstrap pipeline flow, (*bottom-right*) top-10 critical files by PageRank. Together these three panels capture the full structural picture of Claude Code.

V. Discussion

A. Architectural Brittleness at the Core

The 15% import concentration through `models` → `commands` → `tools` represents a classical *God Object* anti-pattern [11] at the module level. While this centralization simplifies routing logic, it creates a fragile axis: any breaking change to `models` propagates immediately to both `commands` and `tools`, affecting the entirety of the command and tool dispatch surfaces.

From an evolutionary architecture perspective [12], decoupling this axis would be the highest-leverage refactoring opportunity in the codebase.

B. Memory as a Finite Resource

The `compact(keep_last=10)` mechanism represents a deliberate *finite memory contract*. Rather than relying on the LLM's native context window, the system enforces its own eviction policy at the application layer:

1) Claude's apparent forgetfulness in long sessions is a software policy, not a model capability limitation.
2) The threshold (10 turns) can theoretically be tuned.
3) Session persistence via `session_store.py` coexists with in-memory eviction, suggesting sessions are recoverable after compaction via disk lookup.

C. Bootstrap Pipeline Robustness

The 7-stage sequential pipeline is a fail-fast design: any stage failure aborts immediately. While this simplifies error propagation, it means configuration errors (Stage 1) and dependency resolution failures (Stage 3) produce the same user-facing symptom: a non-starting agent. Improved error messages distinguishing pipeline stage failures would significantly aid diagnostics.

D. Feature Flag Minimalism

With only 4 feature flags, Claude Code occupies the minimal end of the *configurability spectrum* [13]. LLM agents with many configurable toggles introduce combinatorial behavior spaces that are difficult to test and reason about. Claude Code instead hardcodes behavior, trading configurability for predictability — a defensible choice for a production AI tool.

E. Utility Subsystem Dominance

The `utils` (564 modules) and `components` (389 modules) subsystems account for 50% of the TypeScript codebase. This is consistent with mature systems where foundational infrastructure outweighs feature code, and suggests Claude Code's extensibility surface is primarily component-based rather than service-based.

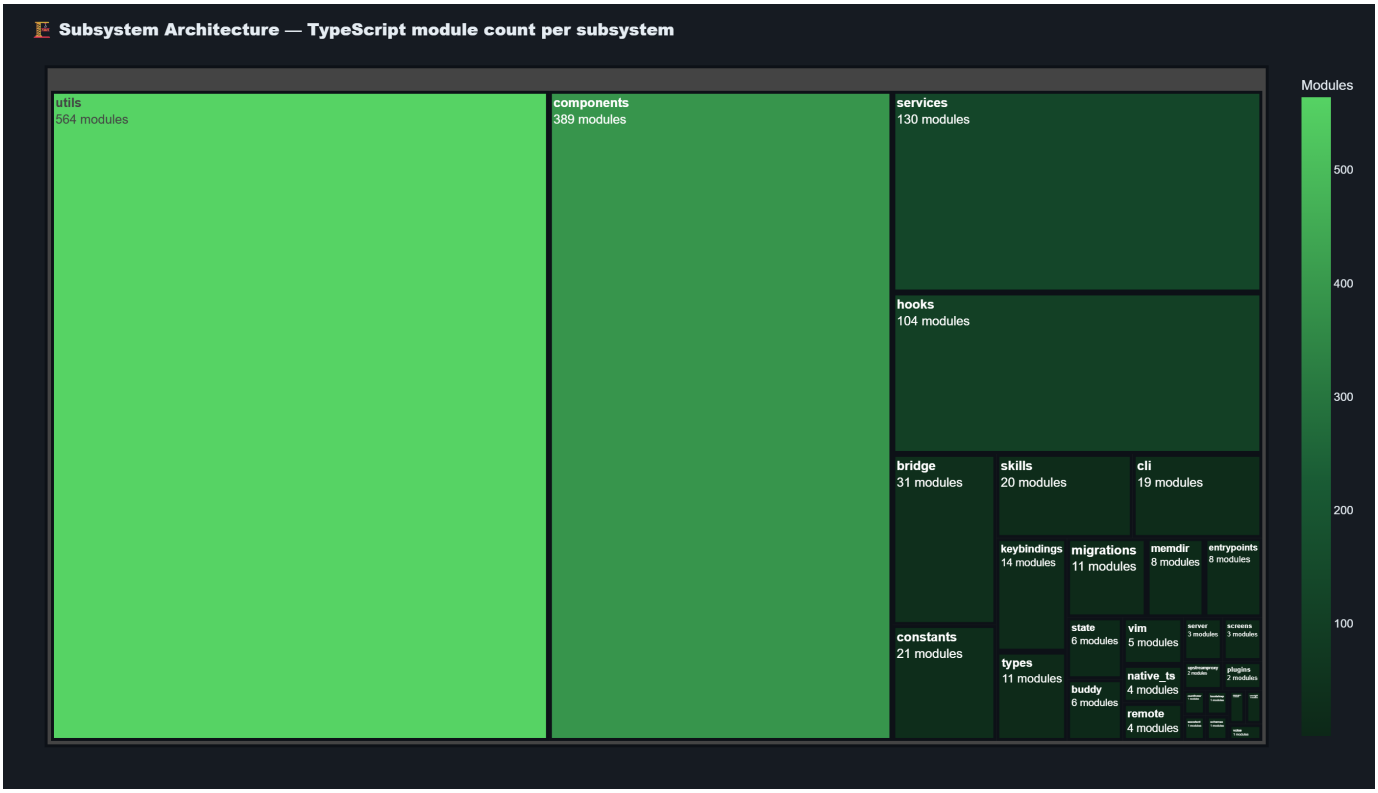

Fig. 10. Subsystem architecture treemap across all 29 subsystems. Area $\propto$ module count. `utils` (564) and `components` (389) together represent 50% of the original TypeScript codebase.

VI. Threats to Validity

A. Internal Validity

The analysis targets `claw-code`, a Python *port* of Claude Code, not the original TypeScript source. Porting may have introduced structural changes not present in the original architecture. String stripping, comment removal, and import reorganization could affect our findings.

B. External Validity

Our findings apply to the version of `claw-code` available at analysis time (April 2026). Anthropic may have shipped architectural changes in newer Claude Code versions. The `compact()` threshold, pipeline stages, and feature flags should be treated as version-specific findings.

C. Construct Validity

PageRank centrality measures *import-graph influence*, a proxy for architectural criticality but not a direct measure of runtime importance. A module with high PageRank might be critical at load time but rarely executed at runtime, and vice versa.

VII. Conclusion

We presented *Claude X-Ray*, a static analysis study of the internal architecture of Claude Code via its open Python port `claw-code`. Our analysis reveals:

- A 15% import bottleneck through `models` $\rightarrow$ `commands` $\rightarrow$ `tools` creating a systemic fragility axis.
- An explicit memory eviction policy (`compact(keep_last=10)`) that explains Claude's observed long-session behavior.
- A strict 7-stage bootstrap pipeline where any initialization failure is catastrophic.
- A minimal feature-flag surface (4 flags) reflecting a deliberate design philosophy of predictability over configurability.
- `query_engine` as the highest-PageRank node and de facto architectural center of the system.

These findings are actionable: developers extending Claude Code should treat the `models/commands/tools` axis as a fragile zone; users experiencing long-session degradation should understand it as an intentional policy; researchers studying LLM agent design can use Claude Code as a case study in minimalist, deterministic agent architecture.

Future work includes dynamic analysis (runtime profiling of the bootstrap pipeline), comparative analysis against LangChain and AutoGen, and architectural evolution tracking as Claude Code versions are released.

Appendix

All analyses were performed in a Jupyter notebook (`claude_xray.ipynb`) using: `networkx` (graph analysis), `plotly` (visualization), `pandas` (tabular data), and Python's

built-in `ast` module (AST parsing). The notebook is available alongside this paper. The data source (`claw-code`) is publicly available at https://github.com/instructkr/claw-code.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.

[2] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[3] Anthropic, "Claude Code: Agentic Coding in Your Terminal," *Anthropic Technical Report*, 2024. [Online]. Available: https://docs.anthropic.com/claude/claude-code

[4] instructkr, "claw-code: Python Port of Claude Code Agent Harness," GitHub repository, 2024. [Online]. Available: https://github.com/instructkr/claw-code

[5] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," *Stanford InfoLab Technical Report*, 1999.

[6] G. Bavota, A. De Lucia, A. Marcus, and R. Oliveto, "Automating Extract Class Refactoring: An Improved Method and its Evaluation," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1617–1664, 2014.

[7] T. Zhang, C. Gao, L. Ma, M. Lyu, and M. Kim, "An Empirical Study of Common Challenges in Developing Deep Learning Applications," *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2020.

[8] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," *arXiv preprint arXiv:2211.09527*, 2022.

[9] LangChain, "LangChain: Building Applications with LLMs through Composability," GitHub repository, 2023. [Online]. Available: https://github.com/langchain-ai/langchain

[10] Significant Gravitas, "AutoGPT: An Autonomous GPT-4 Experiment," GitHub repository, 2023. [Online]. Available: https://github.com/Significant-Gravitas/AutoGPT

[11] W. H. Brown, R. C. Malveau, H. W. McCormick, and T. J. Mowbray, *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. New York: Wiley, 1998.

[12] N. Ford, R. Parsons, and P. Kua, *Building Evolutionary Architectures*. Sebastopol, CA: O'Reilly Media, 2017.

[13] M. Neri and A. Carzaniga, "Software Variability through Configuration," *Proc. ICSE Workshop on Variability Modelling of Software-Intensive Systems*, 2013.